Министерство образования Республики Беларусь
Учреждение образования
«Брестский государственный технический университет»
Кафедра ИИТ

Лабораторная работа №4

По дисциплине: «Модели решения задач в интеллектуальных системах»

Тема: «Многослойный персептрон для прогнозирования временного ряда: влияние размера окна и числа скрытых нейронов»

Выполнила:
Студент 4 курса
Группы ИИ-26
Карпович К.А.
Проверил:
Никитин С.К.

Брест 2026

Цель работы: на основе многослойного персептрона p–N–1 решить задачу прогнозирования временного ряда. Научиться формировать обучающую выборку методом скользящего окна, нормализовать данные, оценивать качество одношагового и многошагового прогноза и исследовать влияние размера окна p и числа нейронов скрытого слоя N на точность прогноза и скорость обучения.

Задачи лабораторной работы

1. Сгенерировать ряд по формуле варианта и построить его график с отметкой границы между обучающим и тестовым участками.
2. Нормализовать ряд по минимуму и максимуму **обучающего участка**: $x_norm = (x - x_min) / (x_max - x_min)$. Записать формулу обратного пересчёта и использовать её при вычислении всех метрик в исходной шкале. **Объяснить**, почему min/max нельзя брать по всему ряду.
3. Сформировать обучающую и тестовую выборки методом скользящего окна размера p (вход – p значений, цель – следующее значение). Указать количество получившихся примеров в каждой выборке.
4. Реализовать многослойный персептрон p–N–1:
   - скрытый слой – сигмоида;
   - выходной слой – линейный (задача регрессии), f(s) = s;
   - функция потерь – MSE;
   - обучение – онлайн-режим, перемешивание порядка примеров в каждой эпохе;
   - вывести формулы градиентов для линейного выходного слоя и сигмоидного скрытого слоя и привести их в отчёте.
5. Критерий остановки: средняя по обучающей выборке ошибка Es ≤ Ee, где Ee = 10^{-4} (нормализованная шкала), либо достижение 20 000 эпох. Скорость обучения подобрать самостоятельно из диапазона 0.01–0.3 и обосновать выбор.
6. **Базовый эксперимент.** Обучить сеть с параметрами p и N из своего варианта. Зафиксировать число эпох, итоговую Es, а также метрики на обучающем и тестовом участках **в исходной шкале**: MAE, RMSE и MAPE (если ряд проходит через ноль – вместо MAPE использовать sMAPE, и пояснить почему).
7. **Исследование окна.** При фиксированном N из варианта обучить сеть для p ∈ {2, 4, p_вар, 2·p_вар}. Для каждого p выполнить по 5 запусков с разными seed и свести в таблицу: среднее и разброс числа эпох, среднее и разброс тестового MAE, число запусков, не достигших критерия остановки.
8. **Исследование скрытого слоя.** При фиксированном p из варианта обучить сеть для N ∈ {1, 2, N_вар, 2·N_вар}. Выполнить ту же серию из 5 запусков и заполнить аналогичную таблицу.
9. **Многошаговый прогноз.** Для лучшей по тестовому MAE конфигурации выполнить рекурсивный прогноз на 30 шагов вперёд, начиная с k = 140: каждое предсказание подставляется в окно как входное значение для следующего шага. Построить график «эталон – рекурсивный прогноз» и график абсолютной ошибки в зависимости от номера шага. Определить, на каком шаге ошибка превышает 5 % от размаха ряда.
10. Построить графики:
    - ряд с границей train/test и одношаговым прогнозом сети на обоих участках на одних осях;
    - сходимость (Es от эпохи) для базовой конфигурации и для двух конфигураций из исследования окна на одних осях (логарифмическая шкала по Y);
    - диаграмма рассеяния «эталон – прогноз» на тестовом участке с диагональю y = x;
    - рекурсивный прогноз и рост ошибки по шагам (п. 9).
11. Реализовать режим функционирования: пользователь вводит p последовательных значений ряда в исходной шкале (либо номер k, с которого брать окно из ряда). Программа выводит:
    - нормализованные входы;
    - прогноз в нормализованной шкале (4 знака после запятой);
    - прогноз в исходной шкале;
    - истинное значение x(k) и абсолютную ошибку (если окно взято из ряда).

Продемонстрировать работу не менее чем на 3 окнах из обучающего участка, 3 окнах из тестового участка и 1–2 пользовательских окнах (в том числе с «выбросом»).

12. Вставить в отчет фото/скан рукописных расчётов. Вверху листа от руки должны быть написаны Ваша Фамилия, Инициалы, номер группы, вариант и дата выполнения:
    - Графическая схема пайплайна (Блок-схема). Составьте корректную цепочку обработки данных, используя предложенный набор блоков (приложение 1). Объясните свой выбор.
    - Ручной расчёт 1 шага обучения. На отдельном листе бумаги от руки распишите один полный шаг (Forward + Backward pass) для сети 3–2–1 со следующими параметрами, привязанными к вашему варианту V:

- Входы: X = [0.2, 0.5, 0.8], Цель: d = 0.6
- Веса скрытого слоя: W11 = 0.05*V, W12 = -0.2, W21 = 0.3, W22 = 0.05*V, W31 = -0.1, W32 = 0.4 (b1=0.1, b2=-0.1)
- Веса выходного слоя: W_out1 = 0.5, W_out2 = -0.2*V (b_out = 0.2)

13. Сделать вывод: ответить на все вопросы ниже и проанализировать результаты.

Требования к реализации

- Скрытый слой – сигмоида, выходной слой – линейный. Веса инициализируются малыми случайными значениями, для воспроизводимости использовать фиксированный seed.
- Обучение – алгоритм обратного распространения в онлайн-режиме.
- Допускается использование NumPy и Matplotlib.
- Запрещено использование scikit-learn, PyTorch, TensorFlow и других ML-фреймворков, а также готовых функций для скользящего окна и расчёта метрик из внешних библиотек (кроме NumPy).
- Код должен быть структурирован: отдельные функции для генерации ряда, нормализации, построения окон, прямого и обратного проходов, обучения, метрик, рекурсивного прогноза и режима функционирования.

Ряд: $x(k)=a\cdot cos(b\cdot k)+c\cdot sin(d\cdot k)$,
k = 0…199, p – размер окна, N – число скрытых нейронов.

Вариант	a	b	c	d	p	N
6	1.2	0.28	0.6	0.65	6	4

- Обучающий участок: k = 0…139 (70 %), тестовый: k = 140…199 (30 %).
- Условия обучения: скрытый слой сигмоидный, выход линейный, функция потерь MSE, онлайн-режим с перемешиванием в каждой эпохе.
- Остановка: Es ≤ Ee = 10^{-4} или 20 000 эпох.

Описание метода

Нормализация (только по train):

```
x_norm = (x − x_min) / (x_max − x_min)
x = x_norm · (x_max − x_min) + x_min
x_min = −1.784001,  x_max = 1.782970
```

min/max нельзя брать по всему ряду: тестовые данные влияют на обучение (утечка информации). В данной задаче будущих значений нет, поэтому масштаб должен определяться только по уже известному участку.

Скользящее окно: вход (x(k−1), …, x(k−p)), цель x(k). При p = 6 получилось 134 обучающих примера (цели k = 6…139) и 60 тестовых (цели k = 140…199).

Архитектура p–N–1:

```
s_j = Σ_i W1_ji · x_i + b1_j ,   h_j = 1/(1 + e^(-s_j))
y   = Σ_j W2_j · h_j + b2        (линейный выход, f(s) = s)
```

Функция потерь: E = (y − d)², Es = среднее E по обучающей выборке.

Вывод градиентов. Обозначим e = y − d.

- Выходной слой (f′ = 1):
  - ∂E/∂y = 2e, поэтому δ_out = 2e;
  - ∂E/∂W2_j = δ_out · h_j, ∂E/∂b2 = δ_out.
- Производная сигмоиды: h′ = h(1 − h).
- Скрытый слой:

- $\delta_j = \delta_{out} \cdot W2_j \cdot h_j(1 - h_j)$;
- $\partial E/\partial W1_{ji} = \delta_j \cdot x_i$, $\partial E/\partial b1_j = \delta_j$.

- Обновление: $w \leftarrow w - \eta \cdot \partial E/\partial w$ (онлайн, после каждого примера).

Метрики (в исходной шкале):

$$MAE = (1/n)\ \Sigma\ |x - \hat{x}|$$
$$RMSE = \sqrt{[(1/n)\ \Sigma\ (x - \hat{x})^2]}$$
$$MAPE = (100/n)\ \Sigma\ |(x - \hat{x})/x|$$
$$sMAPE = (100/n)\ \Sigma\ 2|x - \hat{x}| / (|x| + |\hat{x}|)$$

Ряд 18 раз меняет знак и проходит около нуля, знаменатель MAPE становится почти нулевым, и метрика искажается. Поэтому основной метрикой в выводах я использую sMAPE.

Код программы:

```
import argparse
import os
import sys
import time

import numpy as np
import matplotlib.pyplot as plt

A, B, C, D = 1.2, 0.28, 0.6, 0.65
WINDOW_P = 6
HIDDEN_N = 4
HAND_COEF = 6
SERIES_LEN = 200
TRAIN_LEN = 140
EE = 1e-4
MAX_EPOCHS = 20000
SEEDS = (1, 2, 3, 4, 5)
ETA_CANDIDATES = (0.01, 0.05, 0.1, 0.2, 0.3)
HORIZON = 30
OUT_DIR = os.path.join(os.path.dirname(os.path.abspath(__file__)), "lab4_output")
MODEL_PATH = os.path.join(OUT_DIR, "base_model.npz")
REPORT_PATH = os.path.join(OUT_DIR, "report.txt")


class Reporter:
    def __init__(self):
        self.lines = []

    def __call__(self, text=""):
        print(text, flush=True)
        self.lines.append(str(text))

    def save(self, path):
        with open(path, "w", encoding="utf-8") as f:
            f.write("\n".join(self.lines) + "\n")


report = Reporter()


def generate_series(a, b, c, d, n):
    k = np.arange(n)
    return a * np.cos(b * k) + c * np.sin(d * k)


def fit_minmax(train_part):
    return float(np.min(train_part)), float(np.max(train_part))


def normalize(x, x_min, x_max):
    return (np.asarray(x, dtype=float) - x_min) / (x_max - x_min)


def denormalize(x_norm, x_min, x_max):
    return np.asarray(x_norm, dtype=float) * (x_max - x_min) + x_min


def build_windows(series, p, first_target, last_target):
    inputs = []
    targets = []
    for k in range(first_target, last_target + 1):
        inputs.append(series[k - p:k][::-1])
        targets.append(series[k])
    return np.array(inputs), np.array(targets)
```

```
def sigmoid(s):
    return 1.0 / (1.0 + np.exp(-s))


def init_weights(p, n_hidden, seed, scale=0.3):
    rng = np.random.default_rng(seed)
    return {
        "W1": rng.uniform(-scale, scale, (n_hidden, p)),
        "b1": rng.uniform(-scale, scale, n_hidden),
        "W2": rng.uniform(-scale, scale, n_hidden),
        "b2": float(rng.uniform(-scale, scale)),
    }


def forward(params, x):
    h = sigmoid(params["W1"] @ x + params["b1"])
    y = float(params["W2"] @ h + params["b2"])
    return h, y


def backward(params, x, h, y, target):
    delta_out = 2.0 * (y - target)
    delta_hidden = delta_out * params["W2"] * h * (1.0 - h)
    return {
        "W2": delta_out * h,
        "b2": delta_out,
        "W1": np.outer(delta_hidden, x),
        "b1": delta_hidden,
    }


def apply_gradients(params, grads, eta):
    params["W2"] -= eta * grads["W2"]
    params["b2"] -= eta * grads["b2"]
    params["W1"] -= eta * grads["W1"]
    params["b1"] -= eta * grads["b1"]


def predict_batch(params, inputs):
    hidden = sigmoid(inputs @ params["W1"].T + params["b1"])
    return hidden @ params["W2"] + params["b2"]


def mean_squared_error(params, inputs, targets):
    return float(np.mean((predict_batch(params, inputs) - targets) ** 2))


def train_online(inputs, targets, p, n_hidden, eta, seed, max_epochs=MAX_EPOCHS, ee=EE):
    params = init_weights(p, n_hidden, seed)
    rng = np.random.default_rng(seed + 10_000)
    history = []
    count = len(targets)
    converged = False
    for epoch in range(1, max_epochs + 1):
        for i in rng.permutation(count):
            h, y = forward(params, inputs[i])
            grads = backward(params, inputs[i], h, y, targets[i])
            apply_gradients(params, grads, eta)
        es = mean_squared_error(params, inputs, targets)
        history.append(es)
        if es <= ee:
            converged = True
            break
    return params, np.array(history), converged


def mae(y_true, y_pred):
    return float(np.mean(np.abs(y_true - y_pred)))


def rmse(y_true, y_pred):
    return float(np.sqrt(np.mean((y_true - y_pred) ** 2)))


def mape(y_true, y_pred, eps=1e-12):
    return float(np.mean(np.abs((y_true - y_pred) / np.maximum(np.abs(y_true), eps))) * 100.0)


def smape(y_true, y_pred, eps=1e-12):
    denominator = np.maximum(np.abs(y_true) + np.abs(y_pred), eps)
    return float(np.mean(2.0 * np.abs(y_true - y_pred) / denominator) * 100.0)


def make_dataset(series, p, x_min, x_max):
    norm = normalize(series, x_min, x_max)
    x_train, y_train = build_windows(norm, p, p, TRAIN_LEN - 1)
    x_test, y_test = build_windows(norm, p, TRAIN_LEN, SERIES_LEN - 1)
    return norm, x_train, y_train, x_test, y_test
```

```
def evaluate_original_scale(params, x_in, y_norm, x_min, x_max):
    y_true = denormalize(y_norm, x_min, x_max)
    y_pred = denormalize(predict_batch(params, x_in), x_min, x_max)
    return {
        "MAE": mae(y_true, y_pred),
        "RMSE": rmse(y_true, y_pred),
        "MAPE": mape(y_true, y_pred),
        "sMAPE": smape(y_true, y_pred),
    }


def run_configuration(series, p, n_hidden, eta, seed, x_min, x_max, max_epochs=MAX_EPOCHS):
    norm, x_train, y_train, x_test, y_test = make_dataset(series, p, x_min, x_max)
    params, history, converged = train_online(x_train, y_train, p, n_hidden, eta, seed, max_epochs)
    return {
        "p": p,
        "N": n_hidden,
        "seed": seed,
        "params": params,
        "history": history,
        "epochs": len(history),
        "converged": converged,
        "Es": float(history[-1]),
        "train": evaluate_original_scale(params, x_train, y_train, x_min, x_max),
        "test": evaluate_original_scale(params, x_test, y_test, x_min, x_max),
        "n_train": len(y_train),
        "n_test": len(y_test),
    }


def select_eta(series, x_min, x_max, max_epochs):
    report("Подбор скорости обучения (p=%d, N=%d, seed 1..3)" % (WINDOW_P, HIDDEN_N))
    report("%-8s %-22s %-10s %-12s" % ("eta", "эпохи по seed", "не сошлось", "средн. эпох"))
    best_eta = None
    best_mean = None
    for eta in ETA_CANDIDATES:
        runs = [run_configuration(series, WINDOW_P, HIDDEN_N, eta, s, x_min, x_max, max_epochs) for s in (1, 2, 3)]
        epochs = [r["epochs"] for r in runs]
        failed = sum(0 if r["converged"] else 1 for r in runs)
        mean_epochs = float(np.mean(epochs))
        report("%-8s %-22s %-10d %-12.1f" % (eta, str(epochs), failed, mean_epochs))
        if failed == 0 and (best_mean is None or mean_epochs < best_mean):
            best_mean = mean_epochs
            best_eta = eta
    if best_eta is None:
        best_eta = 0.1
    report("Выбрана скорость обучения eta = %s" % best_eta)
    report()
    return best_eta


def study_table(series, configs, eta, x_min, x_max, max_epochs, title, label):
    report(title)
    header = "%-6s %-18s %-20s %-18s %-18s %-12s" % (
        label, "эпохи (ср.±σ)", "эпохи (min..max)", "test MAE (ср.±σ)", "train MAE (ср.)", "не достигли")
    report(header)
    results = {}
    for p, n_hidden, shown in configs:
        runs = []
        for seed in SEEDS:
            t0 = time.time()
            r = run_configuration(series, p, n_hidden, eta, seed, x_min, x_max, max_epochs)
            runs.append(r)
            print("    p=%d N=%d seed=%d: эпох=%d, Es=%.3e, test MAE=%.4f (%.1f с)" % (
                p, n_hidden, seed, r["epochs"], r["Es"], r["test"]["MAE"], time.time() - t0), flush=True)
        epochs = np.array([r["epochs"] for r in runs], dtype=float)
        test_mae = np.array([r["test"]["MAE"] for r in runs])
        train_mae = np.array([r["train"]["MAE"] for r in runs])
        failed = sum(0 if r["converged"] else 1 for r in runs)
        report("%-6s %-18s %-20s %-18s %-18s %-12s" % (
            shown,
            "%.1f ± %.1f" % (epochs.mean(), epochs.std(ddof=1)),
            "%d .. %d" % (epochs.min(), epochs.max()),
            "%.4f ± %.4f" % (test_mae.mean(), test_mae.std(ddof=1)),
            "%.4f" % train_mae.mean(),
            "%d из %d" % (failed, len(runs))))
        results[(p, n_hidden)] = runs
    report()
    return results


def recursive_forecast(params, norm_series, start, steps, p):
    window = list(norm_series[start - p:start][::-1])
    predictions = []
    for _ in range(steps):
        _, y = forward(params, np.array(window))
        predictions.append(y)
        window = [y] + window[:-1]
    return np.array(predictions)
```

```
def first_exceed_step(errors, threshold):
    for i, e in enumerate(errors, start=1):
        if e > threshold:
            return i
    return None


def plot_series_and_forecast(series, norm_series, params, p, x_min, x_max, path):
    k = np.arange(len(series))
    pred = np.full(len(series), np.nan)
    for i in range(p, len(series)):
        _, y = forward(params, norm_series[i - p:i][::-1])
        pred[i] = y
    pred = denormalize(pred, x_min, x_max)
    plt.figure(figsize=(11, 5))
    plt.plot(k, series, "b-", label="эталон x(k)")
    plt.plot(k[p:TRAIN_LEN], pred[p:TRAIN_LEN], "g--", label="прогноз (обучение)")
    plt.plot(k[TRAIN_LEN:], pred[TRAIN_LEN:], "r--", label="прогноз (тест)")
    plt.axvline(TRAIN_LEN - 0.5, color="k", linestyle=":", label="граница train/test")
    plt.xlabel("k")
    plt.ylabel("x(k)")
    plt.title("Ряд и одношаговый прогноз сети %d-%d-1" % (p, params["W2"].shape[0]))
    plt.grid(alpha=0.3)
    plt.legend()
    plt.tight_layout()
    plt.savefig(path, dpi=130)


def plot_raw_series(series, path):
    k = np.arange(len(series))
    plt.figure(figsize=(11, 4.5))
    plt.plot(k[:TRAIN_LEN], series[:TRAIN_LEN], "b-", label="обучающий участок (k=0..139)")
    plt.plot(k[TRAIN_LEN - 1:], series[TRAIN_LEN - 1:], "r-", label="тестовый участок (k=140..199)")
    plt.axvline(TRAIN_LEN - 0.5, color="k", linestyle=":", label="граница train/test")
    plt.xlabel("k")
    plt.ylabel("x(k)")
    plt.title("x(k) = %.1f·cos(%.2f·k) + %.1f·sin(%.2f·k)" % (A, B, C, D))
    plt.grid(alpha=0.3)
    plt.legend()
    plt.tight_layout()
    plt.savefig(path, dpi=130)


def plot_convergence(runs_with_labels, path):
    plt.figure(figsize=(9, 5))
    for label, run in runs_with_labels:
        plt.plot(np.arange(1, len(run["history"]) + 1), run["history"], label=label)
    plt.axhline(EE, color="k", linestyle=":", label="Ee = 1e-4")
    plt.yscale("log")
    plt.xscale("log")
    plt.xlabel("эпоха")
    plt.ylabel("Es (нормализованная шкала)")
    plt.title("Сходимость обучения")
    plt.grid(alpha=0.3, which="both")
    plt.legend()
    plt.tight_layout()
    plt.savefig(path, dpi=130)


def plot_scatter(params, x_test, y_test, x_min, x_max, path):
    y_true = denormalize(y_test, x_min, x_max)
    y_pred = denormalize(predict_batch(params, x_test), x_min, x_max)
    lo = min(y_true.min(), y_pred.min())
    hi = max(y_true.max(), y_pred.max())
    plt.figure(figsize=(5.8, 5.8))
    plt.scatter(y_true, y_pred, s=22, c="tab:red", label="тестовые точки")
    plt.plot([lo, hi], [lo, hi], "k--", label="y = x")
    plt.xlabel("эталон")
    plt.ylabel("прогноз")
    plt.title("Тестовый участок: эталон - прогноз")
    plt.grid(alpha=0.3)
    plt.legend()
    plt.axis("equal")
    plt.tight_layout()
    plt.savefig(path, dpi=130)


def plot_recursive(true_values, predicted, errors, threshold, exceed_step, path):
    steps = np.arange(1, len(true_values) + 1)
    fig, axes = plt.subplots(1, 2, figsize=(13, 4.8))
    axes[0].plot(steps, true_values, "b-o", ms=3, label="эталон")
    axes[0].plot(steps, predicted, "r--s", ms=3, label="рекурсивный прогноз")
    axes[0].set_xlabel("шаг прогноза (k = 140 + шаг - 1)")
    axes[0].set_ylabel("x(k)")
    axes[0].set_title("Рекурсивный прогноз на %d шагов" % len(steps))
    axes[0].grid(alpha=0.3)
    axes[0].legend()
    axes[1].plot(steps, errors, "m-o", ms=3, label="|ошибка|")
    axes[1].axhline(threshold, color="k", linestyle=":", label="5 %% размаха = %.3f" % threshold)
```

```
    if exceed_step is not None:
        axes[1].axvline(exceed_step, color="g", linestyle="--", label="превышение на шаге %d" % exceed_step)
    axes[1].set_xlabel("шаг прогноза")
    axes[1].set_ylabel("абсолютная ошибка")
    axes[1].set_title("Рост ошибки по шагам")
    axes[1].grid(alpha=0.3)
    axes[1].legend()
    fig.tight_layout()
    fig.savefig(path, dpi=130)


def save_model(path, params, p, n_hidden, eta, x_min, x_max):
    np.savez(path, W1=params["W1"], b1=params["b1"], W2=params["W2"], b2=params["b2"],
             p=p, N=n_hidden, eta=eta, x_min=x_min, x_max=x_max)


def load_model(path):
    data = np.load(path)
    params = {"W1": data["W1"], "b1": data["b1"], "W2": data["W2"], "b2": float(data["b2"])}
    return params, int(data["p"]), int(data["N"]), float(data["x_min"]), float(data["x_max"])


def operate(params, p, x_min, x_max, window_original, series=None, k=None):
    window_original = np.asarray(window_original, dtype=float)
    window_norm = normalize(window_original, x_min, x_max)
    _, y_norm = forward(params, window_norm[::-1])
    y_original = float(denormalize(y_norm, x_min, x_max))
    result = {"window_norm": window_norm, "y_norm": y_norm, "y_original": y_original,
              "true": None, "abs_error": None}
    if series is not None and k is not None:
        result["true"] = float(series[k])
        result["abs_error"] = abs(result["true"] - y_original)
    return result


def print_operation(title, result, window_original):
    report("--- %s ---" % title)
    report("Окно (исходная шкала, от старого к новому): " + ", ".join("%.4f" % v for v in window_original))
    report("Нормализованные входы:                      " + ", ".join("%.4f" % v for v in result["window_norm"]))
    report("Прогноз (нормализованная шкала):            %.4f" % result["y_norm"])
    report("Прогноз (исходная шкала):                   %.4f" % result["y_original"])
    if result["true"] is not None:
        report("Истинное значение x(k):                        %.4f" % result["true"])
        report("Абсолютная ошибка:                             %.4f" % result["abs_error"])
    report()


def operate_from_series(params, p, x_min, x_max, series, k):
    window = series[k - p:k]
    result = operate(params, p, x_min, x_max, window, series, k)
    return window, result


def demonstration(params, p, x_min, x_max, series):
    report("=" * 70)
    report("РЕЖИМ ФУНКЦИОНИРОВАНИЯ: демонстрация")
    report("=" * 70)
    for k in (20, 70, 120):
        window, result = operate_from_series(params, p, x_min, x_max, series, k)
        print_operation("Обучающий участок, k = %d" % k, result, window)
    for k in (145, 170, 199):
        window, result = operate_from_series(params, p, x_min, x_max, series, k)
        print_operation("Тестовый участок, k = %d" % k, result, window)
    custom = [1.5, 1.1, 0.4, -0.5, -1.2, -1.6]
    result = operate(params, p, x_min, x_max, custom)
    print_operation("Пользовательское окно", result, custom)
    outlier_window = series[100 - p:100].copy()
    outlier_window[3] = 6.0
    result = operate(params, p, x_min, x_max, outlier_window, series, 100)
    print_operation("Окно k = 100 с выбросом (4-е значение заменено на 6.0)", result, outlier_window)


def parse_numbers(text):
    return [float(t) for t in text.replace(";", " ").replace(",", " ").split()]


def interactive_mode(params, p, x_min, x_max, series):
    report("=" * 70)
    report("РЕЖИМ ФУНКЦИОНИРОВАНИЯ: интерактивный ввод")
    report("=" * 70)
    print("Введите номер k (%d..%d), либо %d значений ряда через пробел/запятую (исходная шкала)." % (
        p, len(series) - 1, p))
    print("Пустая строка, q или exit - выход.")
    while True:
        try:
            text = input("> ").strip()
        except EOFError:
            break
        if text.lower() in ("", "q", "exit", "quit", "выход"):
            break
```

```
        try:
            values = parse_numbers(text)
        except ValueError:
            print("Не удалось разобрать ввод: используйте числа.")
            continue
        if len(values) == 1 and float(values[0]).is_integer():
            k = int(values[0])
            if not (p <= k <= len(series) - 1):
                print("Номер k должен быть в диапазоне %d..%d." % (p, len(series) - 1))
                continue
            window, result = operate_from_series(params, p, x_min, x_max, series, k)
            print_operation("Окно из ряда, k = %d" % k, result, window)
        elif len(values) == p:
            result = operate(params, p, x_min, x_max, values)
            print_operation("Пользовательское окно", result, values)
        else:
            print("Нужно ввести ровно %d чисел или один целый номер k." % p)


def hand_calculation(coef, eta):
    x = np.array([0.2, 0.5, 0.8])
    d = 0.6
    W1 = np.array([[0.05 * coef, 0.3, -0.1],
                   [-0.2, 0.05 * coef, 0.4]])
    b1 = np.array([0.1, -0.1])
    W2 = np.array([0.5, -0.2 * coef])
    b2 = 0.2
    report("=" * 70)
    report("РУЧНОЙ РАСЧЁТ 1 ШАГА (сеть 3-2-1, коэффициент %d, eta = %s) - для самопроверки" % (coef, eta))
    report("=" * 70)
    s = W1 @ x + b1
    h = sigmoid(s)
    y = float(W2 @ h + b2)
    e = y - d
    E = e ** 2
    report("s1 = %.6f, s2 = %.6f" % (s[0], s[1]))
    report("h1 = %.6f, h2 = %.6f" % (h[0], h[1]))
    report("y = %.6f, e = y - d = %.6f, E = e^2 = %.6f" % (y, e, E))
    delta_out = 2 * e
    delta_h = delta_out * W2 * h * (1 - h)
    report("delta_out = 2e = %.6f" % delta_out)
    report("dE/dWout = [%.6f, %.6f], dE/db_out = %.6f" % (delta_out * h[0], delta_out * h[1], delta_out))
    report("delta_h1 = %.6f, delta_h2 = %.6f" % (delta_h[0], delta_h[1]))
    for j in range(2):
        report("dE/dW[:,%d] = [%.6f, %.6f, %.6f], dE/db%d = %.6f" % (
            j + 1, delta_h[j] * x[0], delta_h[j] * x[1], delta_h[j] * x[2], j + 1, delta_h[j]))
    W2n = W2 - eta * delta_out * h
    b2n = b2 - eta * delta_out
    W1n = W1 - eta * np.outer(delta_h, x)
    b1n = b1 - eta * delta_h
    report("Новые Wout = [%.6f, %.6f], b_out = %.6f" % (W2n[0], W2n[1], b2n))
    for j in range(2):
        report("Новые веса нейрона %d: [%.6f, %.6f, %.6f], b%d = %.6f" % (
            j + 1, W1n[j][0], W1n[j][1], W1n[j][2], j + 1, b1n[j]))
    h_new = sigmoid(W1n @ x + b1n)
    y_new = float(W2n @ h_new + b2n)
    report("Выход после шага: %.6f, ошибка %.6f -> %.6f" % (y_new, E, (y_new - d) ** 2))
    report()


def noise_experiment(series, eta, x_min, x_max, max_epochs, sigma=0.02):
    max_epochs = min(max_epochs, 3000)
    report("Эксперимент с шумом (гауссов шум, σ = %.2f в нормализованной шкале, только в обучающем ряду, не более %d эпох)" %
(sigma, max_epochs))
    norm = normalize(series, x_min, x_max)
    x_test, y_test = build_windows(norm, WINDOW_P, TRAIN_LEN, SERIES_LEN - 1)
    rows = []
    for use_noise in (False, True):
        test_maes = []
        train_maes = []
        failed = 0
        for seed in SEEDS:
            train_series = norm.copy()
            if use_noise:
                rng = np.random.default_rng(seed + 500)
                train_series[:TRAIN_LEN] += rng.normal(0.0, sigma, TRAIN_LEN)
            x_tr, y_tr = build_windows(train_series, WINDOW_P, WINDOW_P, TRAIN_LEN - 1)
            params, history, converged = train_online(x_tr, y_tr, WINDOW_P, HIDDEN_N, eta, seed, max_epochs)
            failed += 0 if converged else 1
            clean_train_x, clean_train_y = build_windows(norm, WINDOW_P, WINDOW_P, TRAIN_LEN - 1)
            train_maes.append(mae(denormalize(clean_train_y, x_min, x_max),
                                  denormalize(predict_batch(params, clean_train_x), x_min, x_max)))
            test_maes.append(mae(denormalize(y_test, x_min, x_max),
                                 denormalize(predict_batch(params, x_test), x_min, x_max)))
        rows.append(("с шумом" if use_noise else "без шума", np.mean(train_maes), np.mean(test_maes),
                     np.std(test_maes, ddof=1), failed))
    report("%-12s %-22s %-22s %-14s %-10s" % ("обучение", "train MAE (чистый ряд)", "test MAE (ср.)", "σ test MAE", "не
сошлось"))
    for name, tr, te, sd, failed in rows:
        report("%-12s %-22.4f %-22.4f %-14.4f %-10d" % (name, tr, te, sd, failed))
```

```
    report("(для шумного обучения критерий Es ≤ Ee может быть недостижим, т.к. шум не воспроизводится сетью)")
    report()


def run_all(args):
    os.makedirs(OUT_DIR, exist_ok=True)
    max_epochs = 2000 if args.quick else MAX_EPOCHS
    series = generate_series(A, B, C, D, SERIES_LEN)
    x_min, x_max = fit_minmax(series[:TRAIN_LEN])
    report("Лабораторная работа №4")
    report("x(k) = %.2f*cos(%.2f*k) + %.2f*sin(%.2f*k), k = 0..%d" % (A, B, C, D, SERIES_LEN - 1))
    report("p = %d, N = %d, Ee = %g, макс. эпох = %d" % (WINDOW_P, HIDDEN_N, EE, max_epochs))
    report()
    report("Нормализация по обучающему участку: x_min = %.6f, x_max = %.6f" % (x_min, x_max))
    report("Формула: x_norm = (x - x_min)/(x_max - x_min);  обратно: x = x_norm*(x_max - x_min) + x_min")
    report("Весь ряд: min = %.6f, max = %.6f" % (series.min(), series.max()))
    norm = normalize(series, x_min, x_max)
    outside = np.where((norm < 0) | (norm > 1))[0]
    report("Нормализованный ряд: min = %.6f, max = %.6f, точек вне [0;1]: %d %s" % (
        norm.min(), norm.max(), len(outside), [int(i) for i in outside] if len(outside) else ""))
    zero_cross = int(np.sum(np.sign(series[:-1]) * np.sign(series[1:]) < 0))
    report("Ряд меняет знак %d раз(а) и проходит рядом с нулём -> MAPE неприменима, используется sMAPE" % zero_cross)
    report()

    plot_raw_series(series, os.path.join(OUT_DIR, "01_series.png"))

    _, x_train, y_train, x_test, y_test = make_dataset(series, WINDOW_P, x_min, x_max)
    report("Окна p = %d: обучающих примеров = %d (цели k = %d..%d), тестовых = %d (цели k = %d..%d)" % (
        WINDOW_P, len(y_train), WINDOW_P, TRAIN_LEN - 1, len(y_test), TRAIN_LEN, SERIES_LEN - 1))
    report()

    eta = args.eta if args.eta is not None else select_eta(series, x_min, x_max, max_epochs)
    if args.eta is not None:
        report("Скорость обучения задана вручную: eta = %s" % eta)
        report()

    base = run_configuration(series, WINDOW_P, HIDDEN_N, eta, SEEDS[0], x_min, x_max, max_epochs)
    report("БАЗОВЫЙ ЭКСПЕРИМЕНТ p=%d, N=%d, eta=%s, seed=%d" % (WINDOW_P, HIDDEN_N, eta, SEEDS[0]))
    report("Эпох: %d, достигнут критерий: %s, итоговая Es = %.3e" % (base["epochs"], "да" if base["converged"] else "нет",
base["Es"]))
    report("%-10s %-10s %-10s %-12s %-10s" % ("участок", "MAE", "RMSE", "MAPE, %", "sMAPE, %"))
    for name in ("train", "test"):
        m = base[name]
        report("%-10s %-10.5f %-10.5f %-12.3f %-10.4f" % (
            "обучение" if name == "train" else "тест", m["MAE"], m["RMSE"], m["MAPE"], m["sMAPE"]))
    report()
    save_model(MODEL_PATH, base["params"], WINDOW_P, HIDDEN_N, eta, x_min, x_max)

    windows_p = sorted({2, 4, WINDOW_P, 2 * WINDOW_P})
    configs_p = [(p, HIDDEN_N, str(p)) for p in windows_p]
    results_p = study_table(series, configs_p, eta, x_min, x_max, max_epochs,
                            "ИССЛЕДОВАНИЕ ОКНА (N = %d, 5 запусков, seed 1..5)" % HIDDEN_N, "p")
    hidden_list = sorted({1, 2, HIDDEN_N, 2 * HIDDEN_N})
    configs_n = [(WINDOW_P, n, str(n)) for n in hidden_list]
    results_n = study_table(series, configs_n, eta, x_min, x_max, max_epochs,
                            "ИССЛЕДОВАНИЕ СКРЫТОГО СЛОЯ (p = %d, 5 запусков, seed 1..5)" % WINDOW_P, "N")

    all_results = {}
    all_results.update(results_p)
    all_results.update(results_n)
    best_key = min(all_results, key=lambda key: np.mean([r["test"]["MAE"] for r in all_results[key]]))
    best_run = min(all_results[best_key], key=lambda r: r["test"]["MAE"])
    report("Лучшая конфигурация по среднему тестовому MAE: p=%d, N=%d (ср. test MAE = %.5f); "
           "лучший запуск seed=%d, test MAE = %.5f" % (
               best_key[0], best_key[1], np.mean([r["test"]["MAE"] for r in all_results[best_key]]),
               best_run["seed"], best_run["test"]["MAE"]))
    report()

    bp, bn = best_key
    best_params = best_run["params"]
    steps = HORIZON
    pred_norm = recursive_forecast(best_params, norm, TRAIN_LEN, steps, bp)
    pred_orig = denormalize(pred_norm, x_min, x_max)
    true_orig = series[TRAIN_LEN:TRAIN_LEN + steps]
    errors = np.abs(true_orig - pred_orig)
    full_range = float(series.max() - series.min())
    threshold = 0.05 * full_range
    exceed = first_exceed_step(errors, threshold)
    report("РЕКУРСИВНЫЙ ПРОГНОЗ на %d шагов, начиная с k = %d (конфигурация p=%d, N=%d, seed=%d)" % (
        steps, TRAIN_LEN, bp, bn, best_run["seed"]))
    report("Размах ряда = %.4f, 5 %% размаха = %.4f" % (full_range, threshold))
    report("%-6s %-6s %-12s %-12s %-12s" % ("шаг", "k", "эталон", "прогноз", "|ошибка|"))
    for i in range(steps):
        report("%-6d %-6d %-12.4f %-12.4f %-12.4f" % (i + 1, TRAIN_LEN + i, true_orig[i], pred_orig[i], errors[i]))
    if exceed is None:
        report("Ошибка не превышает 5 %% размаха на всём горизонте (макс. ошибка = %.4f)" % errors.max())
    else:
        report("Ошибка впервые превышает 5 %% размаха на шаге %d (k = %d, ошибка %.4f)" % (
            exceed, TRAIN_LEN + exceed - 1, errors[exceed - 1]))
    one_step = np.abs(denormalize(build_windows(norm, bp, TRAIN_LEN, TRAIN_LEN + steps - 1)[1], x_min, x_max)
```

```
                    - denormalize(predict_batch(best_params, build_windows(norm, bp, TRAIN_LEN, TRAIN_LEN + steps - 1)[0]),
x_min, x_max))
    report("Для сравнения: средняя ошибка одношагового прогноза на тех же %d точках = %.5f, рекурсивного = %.5f" % (
        steps, one_step.mean(), errors.mean()))
    report()

    plot_series_and_forecast(series, norm, base["params"], WINDOW_P, x_min, x_max, os.path.join(OUT_DIR, "02_forecast.png"))
    labels = [("база p=%d, N=%d" % (WINDOW_P, HIDDEN_N), base)]
    for p_extra in (2, 2 * WINDOW_P):
        labels.append(("p=%d, N=%d" % (p_extra, HIDDEN_N), results_p[(p_extra, HIDDEN_N)][0]))
    plot_convergence(labels, os.path.join(OUT_DIR, "03_convergence.png"))
    _, _, _, x_te_best, y_te_best = make_dataset(series, bp, x_min, x_max)
    plot_scatter(best_params, x_te_best, y_te_best, x_min, x_max, os.path.join(OUT_DIR, "04_scatter.png"))
    plot_recursive(true_orig, pred_orig, errors, threshold, exceed, os.path.join(OUT_DIR, "05_recursive.png"))

    noise_experiment(series, eta, x_min, x_max, max_epochs)
    hand_calculation(HAND_COEF, 0.1)
    demonstration(base["params"], WINDOW_P, x_min, x_max, series)
    report.save(REPORT_PATH)
    report("Графики и report.txt сохранены в папку: %s" % OUT_DIR)
    return base["params"], series, x_min, x_max, eta


def parse_args(argv):
    parser = argparse.ArgumentParser(description="ЛР №4: MLP p-N-1 для прогнозирования временного ряда")
    parser.add_argument("--quick", action="store_true", help="быстрый тест: максимум 2000 эпох")
    parser.add_argument("--eta", type=float, default=None, help="задать скорость обучения вручную")
    parser.add_argument("--no-show", action="store_true", help="не открывать окна с графиками")
    parser.add_argument("--no-interactive", action="store_true", help="не запускать интерактивный режим")
    parser.add_argument("--operate-only", action="store_true", help="только режим функционирования (нужен сохранённый model)")
    return parser.parse_args(argv)


def main(argv=None):
    args = parse_args(sys.argv[1:] if argv is None else argv)
    if args.no_show:
        plt.switch_backend("Agg")
    if args.operate_only and os.path.exists(MODEL_PATH):
        params, p, n_hidden, x_min, x_max = load_model(MODEL_PATH)
        series = generate_series(A, B, C, D, SERIES_LEN)
        interactive_mode(params, p, x_min, x_max, series)
        return
    params, series, x_min, x_max, eta = run_all(args)
    if not args.no_show:
        plt.show()
    if not args.no_interactive:
        interactive_mode(params, WINDOW_P, x_min, x_max, series)


if __name__ == "__main__":
    main()
```

Результат программы:

```
x(k) = 1.20*cos(0.28*k) + 0.60*sin(0.65*k), k = 0..199
p = 6, N = 4, Ee = 0.0001, макс. эпох = 20000

Нормализация по обучающему участку: x_min = -1.784001, x_max = 1.782970
Формула: x_norm = (x - x_min)/(x_max - x_min);  обратно: x = x_norm*(x_max - x_min) + x_min
Весь ряд: min = -1.795833, max = 1.798898
Нормализованный ряд: min = -0.003317, max = 1.004465, точек вне [0;1]: 2 [157, 191]
Ряд меняет знак 18 раз(а) и проходит рядом с нулём -> MAPE неприменима, используется sMAPE

Окна p = 6: обучающих примеров = 134 (цели k = 6..139), тестовых = 60 (цели k = 140..199)

Подбор скорости обучения (p=6, N=4, seed 1..3)
eta      эпохи по seed          не сошлось средн. эпох
0.01     [1918, 2044, 2047]     0          2003.0
0.05     [397, 425, 428]        0          416.7
0.1      [211, 232, 231]        0          224.7
0.2      [133, 140, 146]        0          139.7
0.3      [118, 132, 124]        0          124.7
Выбрана скорость обучения eta = 0.3

БАЗОВЫЙ ЭКСПЕРИМЕНТ p=6, N=4, eta=0.3, seed=1
Эпох: 118, достигнут критерий: да, итоговая Es = 8.609e-05
участок    MAE        RMSE       MAPE, %      sMAPE, %
обучение   0.02809    0.03310    7.297        7.6721
тест       0.02766    0.03303    10.437       12.1364

ИССЛЕДОВАНИЕ ОКНА (N = 4, 5 запусков, seed 1..5)
p      эпохи (ср.±σ)      эпохи (min..max)     test MAE (ср.±σ)   train MAE (ср.)    не достигли
2      20000.0 ± 0.0      20000 .. 20000       0.1072 ± 0.0138    0.1110             5 из 5
4      2393.0 ± 71.1      2313 .. 2479         0.0268 ± 0.0008    0.0284             0 из 5
6      131.6 ± 11.1       118 .. 146           0.0273 ± 0.0009    0.0277             0 из 5
12     8.4 ± 2.5          6 .. 12              0.0236 ± 0.0029    0.0236             0 из 5

ИССЛЕДОВАНИЕ СКРЫТОГО СЛОЯ (p = 6, 5 запусков, seed 1..5)
N      эпохи (ср.±σ)      эпохи (min..max)     test MAE (ср.±σ)   train MAE (ср.)    не достигли
```

1	131.4 ± 10.5	119 .. 146	0.0265 ± 0.0005	0.0279	0 из 5
2	138.0 ± 8.4	125 .. 146	0.0255 ± 0.0027	0.0264	0 из 5
4	131.6 ± 11.1	118 .. 146	0.0273 ± 0.0009	0.0277	0 из 5
8	137.4 ± 8.1	127 .. 147	0.0270 ± 0.0021	0.0275	0 из 5

Лучшая конфигурация по среднему тестовому MAE: p=12, N=4 (ср. test MAE = 0.02358); лучший запуск seed=1, test MAE = 0.01919

РЕКУРСИВНЫЙ ПРОГНОЗ на 30 шагов, начиная с k = 140 (конфигурация p=12, N=4, seed=1)
Размах ряда = 3.5947, 5 % размаха = 0.1797

шаг	k	эталон	прогноз	|ошибка|
1	140	0.1474	0.1399	0.0075
2	141	-0.5607	-0.5692	0.0085
3	142	-1.1227	-1.1253	0.0026
4	143	-1.4132	-1.4006	0.0126
5	144	-1.4030	-1.3850	0.0181
6	145	-1.1641	-1.1537	0.0104
7	146	-0.8349	-0.8362	0.0013
8	147	-0.5609	-0.5740	0.0131
9	148	-0.4340	-0.4650	0.0310
10	149	-0.4571	-0.5132	0.0561
11	150	-0.5462	-0.6272	0.0810
12	151	-0.5710	-0.6661	0.0952
13	152	-0.4148	-0.5052	0.0904
14	153	-0.0308	-0.0906	0.0597
15	154	0.5302	0.5257	0.0044
16	155	1.1326	1.1676	0.0350
17	156	1.6038	1.6246	0.0208
18	157	1.7989	1.7717	0.0272
19	158	1.6562	1.5920	0.0642
20	159	1.2209	1.1392	0.0817
21	160	0.6277	0.5388	0.0889
22	161	0.0493	-0.0243	0.0736
23	162	-0.3688	-0.3875	0.0188
24	163	-0.5605	-0.4998	0.0607
25	164	-0.5580	-0.4287	0.1293
26	165	-0.4700	-0.3105	0.1594
27	166	-0.4289	-0.2864	0.1425
28	167	-0.5296	-0.4422	0.0874
29	168	-0.7843	-0.7743	0.0100
30	169	-1.1137	-1.1791	0.0655

Ошибка не превышает 5 % размаха на всём горизонте (макс. ошибка = 0.1594)
Для сравнения: средняя ошибка одношагового прогноза на тех же 30 точках = 0.02110, рекурсивного = 0.05189

Эксперимент с шумом (гауссов шум, σ = 0.02 в нормализованной шкале, только в обучающем ряду, не более 3000 эпох)

обучение	train MAE (чистый ряд)	test MAE (ср.)	σ test MAE	не сошлось
без шума	0.0277	0.0273	0.0009	0
с шумом	0.1158	0.1156	0.0252	5

(для шумного обучения критерий Es ≤ Ee может быть недостижим, т.к. шум не воспроизводится сетью)

```
======================================================================
РУЧНОЙ РАСЧЁТ 1 ШАГА (сеть 3-2-1, коэффициент 6, eta = 0.1) - для самопроверки
======================================================================
s1 = 0.230000, s2 = 0.330000
h1 = 0.557248, h2 = 0.581759
y = -0.219487, e = y - d = -0.819487, E = e^2 = 0.671559
delta_out = 2e = -1.638975
dE/dWout = [-0.913315, -0.953489], dE/db_out = -1.638975
delta_h1 = -0.202186, delta_h2 = 0.478545
dE/dW[:,1] = [-0.040437, -0.101093, -0.161749], dE/db1 = -0.202186
dE/dW[:,2] = [0.095709, 0.239273, 0.382836], dE/db2 = 0.478545
Новые Wout = [0.591332, -1.104651], b_out = 0.363897
Новые веса нейрона 1: [0.304044, 0.310109, -0.083825], b1 = 0.120219
Новые веса нейрона 2: [-0.209571, 0.276073, 0.361716], b2 = -0.147855
Выход после шага: 0.081449, ошибка 0.671559 -> 0.268895

======================================================================
РЕЖИМ ФУНКЦИОНИРОВАНИЯ: демонстрация
======================================================================
--- Обучающий участок, k = 20 ---
Окно (исходная шкала, от старого к новому): -0.6630, -0.7800, -0.7731, -0.5420, -0.0711, 0.5563
Нормализованные входы:                      0.3143, 0.2815, 0.2834, 0.3482, 0.4802, 0.6561
Прогноз (нормализованная шкала):            0.8319
Прогноз (исходная шкала):                   1.1833
Истинное значение x(k):                     1.1828
Абсолютная ошибка:                          0.0006

--- Обучающий участок, k = 70 ---
Окно (исходная шкала, от старого к новому): 0.3048, 0.3634, 0.5892, 0.9438, 1.3079, 1.5273
Нормализованные входы:                      0.5856, 0.6020, 0.6653, 0.7647, 0.8668, 0.9283
Прогноз (нормализованная шкала):            0.9224
Прогноз (исходная шкала):                   1.5062
Истинное значение x(k):                     1.4768
Абсолютная ошибка:                          0.0294

--- Обучающий участок, k = 120 ---
Окно (исходная шкала, от старого к новому): 0.4729, 0.4874, 0.5834, 0.6337, 0.5145, 0.1644
Нормализованные входы:                      0.6327, 0.6368, 0.6637, 0.6778, 0.6444, 0.5462
Прогноз (нормализованная шкала):            0.4097
Прогноз (исходная шкала):                   -0.3225
Истинное значение x(k):                     -0.3823
```

```
Абсолютная ошибка:                          0.0598

--- Тестовый участок, k = 145 ---
Окно (исходная шкала, от старого к новому): 0.8231, 0.1474, -0.5607, -1.1227, -1.4132, -1.4030
Нормализованные входы:                      0.7309, 0.5415, 0.3430, 0.1854, 0.1039, 0.1068
Прогноз (нормализованная шкала):            0.1747
Прогноз (исходная шкала):                   -1.1609
Истинное значение x(k):                     -1.1641
Абсолютная ошибка:                          0.0031

--- Тестовый участок, k = 170 ---
Окно (исходная шкала, от старого к новому): -0.5580, -0.4700, -0.4289, -0.5296, -0.7843, -1.1137
Нормализованные входы:                      0.3437, 0.3684, 0.3799, 0.3517, 0.2803, 0.1879
Прогноз (нормализованная шкала):            0.1201
Прогноз (исходная шкала):                   -1.3557
Истинное значение x(k):                     -1.3772
Абсолютная ошибка:                          0.0216

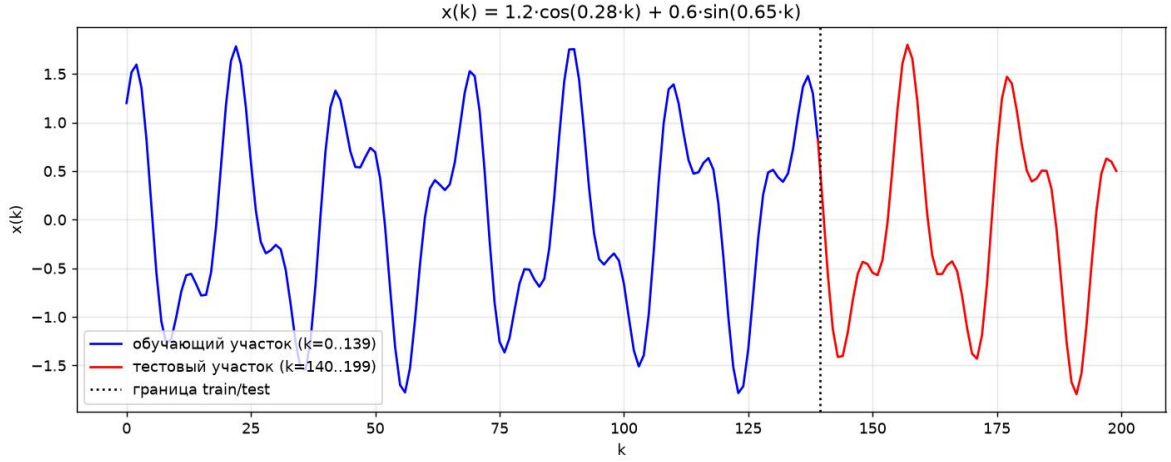
--- Тестовый участок, k = 199 ---
Окно (исходная шкала, от старого к новому): -1.0948, -0.4802, 0.0882, 0.4745, 0.6287, 0.5981
Нормализованные входы:                      0.1932, 0.3655, 0.5249, 0.6332, 0.6764, 0.6678
Прогноз (нормализованная шкала):            0.6413
Прогноз (исходная шкала):                   0.5035
Истинное значение x(k):                     0.5001
Абсолютная ошибка:                          0.0033

--- Пользовательское окно ---
Окно (исходная шкала, от старого к новому): 1.5000, 1.1000, 0.4000, -0.5000, -1.2000, -1.6000
Нормализованные входы:                      0.9207, 0.8085, 0.6123, 0.3600, 0.1637, 0.0516
Прогноз (нормализованная шкала):            0.0559
Прогноз (исходная шкала):                   -1.5846

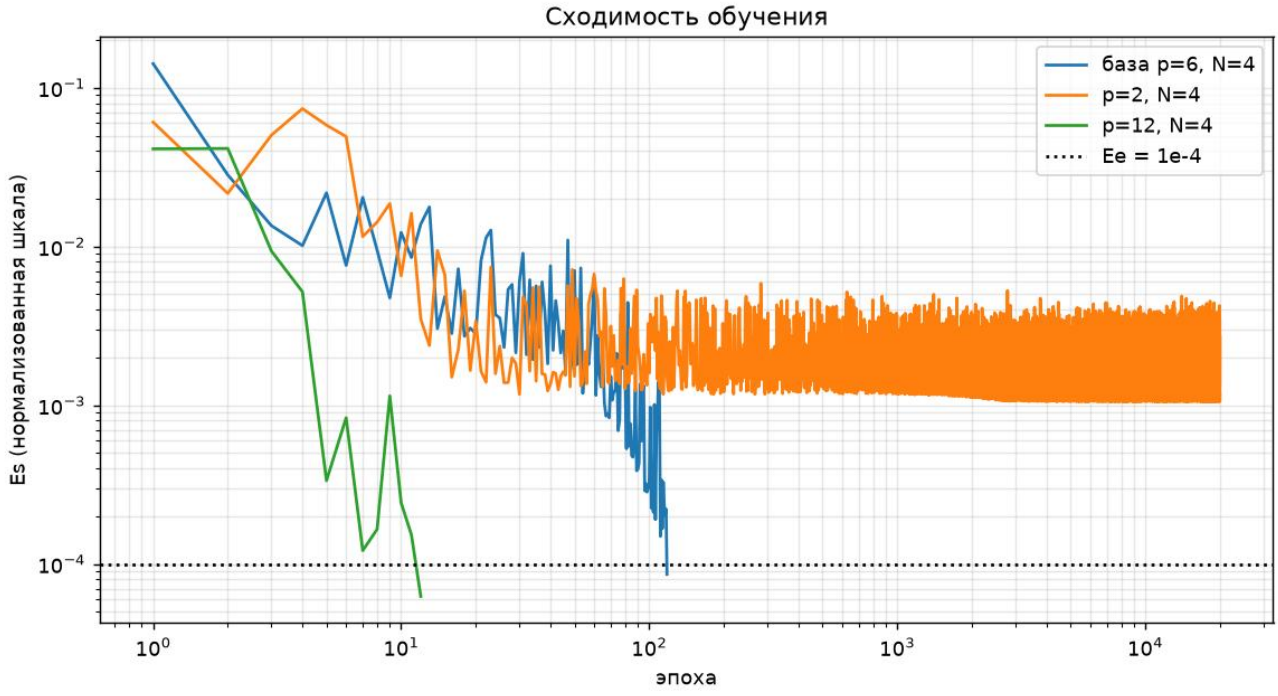
--- Окно k = 100 с выбросом (4-е значение заменено на 6.0) ---
Окно (исходная шкала, от старого к новому): -0.1432, -0.4057, -0.4618, 6.0000, -0.3481, -0.4211
Нормализованные входы:                      0.4600, 0.3864, 0.3707, 2.1822, 0.4025, 0.3821
Прогноз (нормализованная шкала):            -0.9255
Прогноз (исходная шкала):                   -5.0851
Истинное значение x(k):                     -0.6590
Абсолютная ошибка:                          4.4261
```

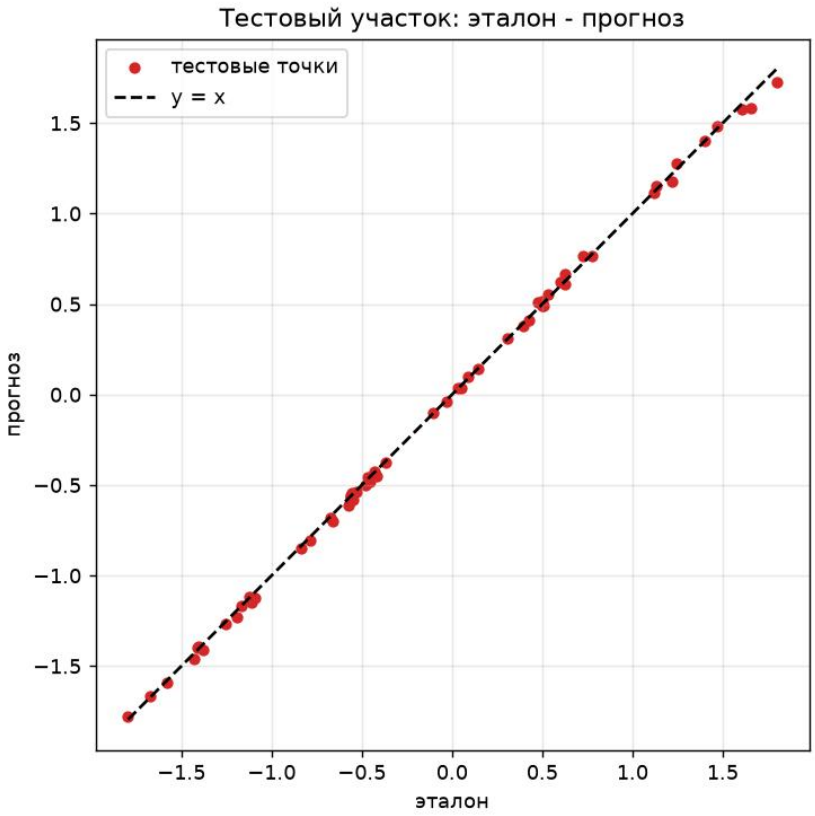
```
============================================================================
РЕЖИМ ФУНКЦИОНИРОВАНИЯ: интерактивный ввод
============================================================================
Введите номер k (6..199), либо 6 значений ряда через пробел/запятую (исходная шкала).
Пустая строка, q или exit - выход.
> 70
--- Окно из ряда, k = 70 ---
Окно (исходная шкала, от старого к новому): 0.3048, 0.3634, 0.5892, 0.9438, 1.3079, 1.5273
Нормализованные входы:                      0.5856, 0.6020, 0.6653, 0.7647, 0.8668, 0.9283
Прогноз (нормализованная шкала):            0.9224
Прогноз (исходная шкала):                   1.5062
Истинное значение x(k):                     1.4768
Абсолютная ошибка:                          0.0294

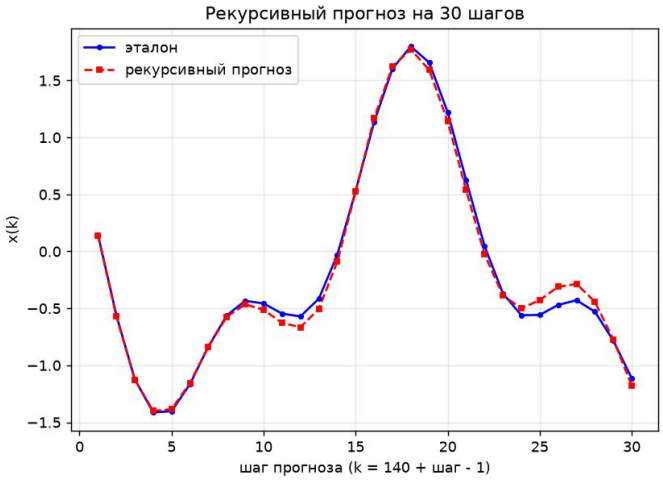
> 145
--- Окно из ряда, k = 145 ---
Окно (исходная шкала, от старого к новому): 0.8231, 0.1474, -0.5607, -1.1227, -1.4132, -1.4030
Нормализованные входы:                      0.7309, 0.5415, 0.3430, 0.1854, 0.1039, 0.1068
Прогноз (нормализованная шкала):            0.1747
Прогноз (исходная шкала):                   -1.1609
Истинное значение x(k):                     -1.1641
Абсолютная ошибка:                          0.0031
```


x(k) = 1.2·cos(0.28·k) + 0.6·sin(0.65·k)
x(k)
k
обучающий участок (k=0..139)
тестовый участок (k=140..199)
граница train/test


Ряд и одношаговый прогноз сети 6-4-1
x(k)
k
эталон x(k)
прогноз (обучение)
прогноз (тест)
граница train/test


Сходимость обучения
Es (нормализованная шкала)
эпоха
база p=6, N=4
p=2, N=4
p=12, N=4
Ee = 1e-4





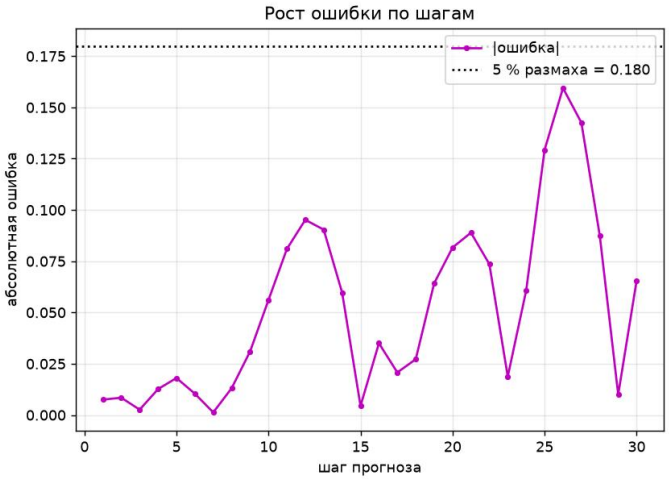
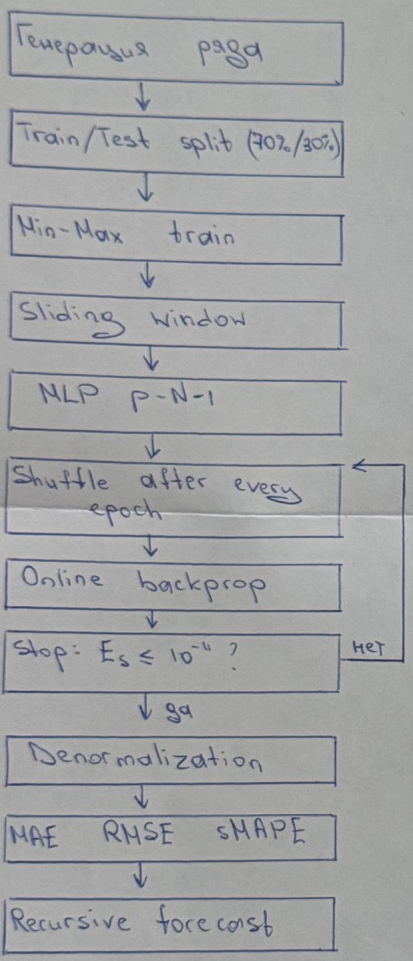


Фото рукописных расчётов:

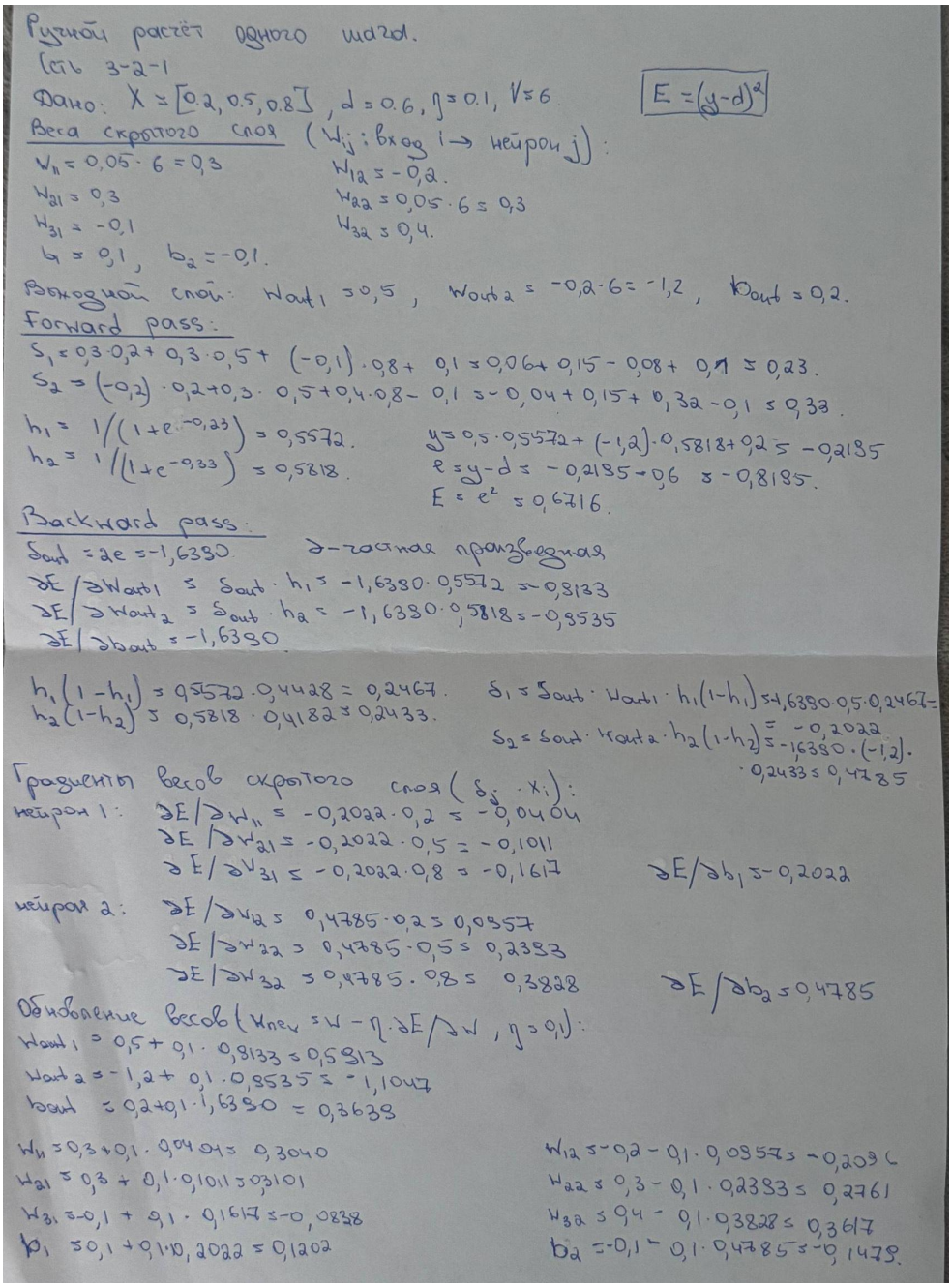
Карпович К.А.
студент группы ИИ-26
Вариант 6
13.10.2026

Графическая схема пайплайна (блок-схема):



Пояснение к выбору блоков:

- Min-Max считается по train до окон, чтобы тест не влиял на обучение
- Shuffle стоит перед backprop, потому что порядок примеров меняется в начале каждой эпохи.
- Метрики считаются после денормализации, чтобы ошибка была в исходной шкале.
- Не использованы блоки: Normalize using whole dataset (утечка), Sigmoid output (выход должен быть линейным), Batch update (обучение онлайн), MAPE (ряд проходит через ноль, берем sMAPE).



Вывод о проделанной работе:

1. Как размер окна p влияет на качество прогноза? При каком p ошибка перестаёт уменьшаться? Почему слишком маленькое окно (p = 2) не позволяет точно описать ряд из двух гармоник, а слишком большое не даёт выигрыша?

- При p = 2 сеть не сходится (Es ≈ 10^{-3}, тестовая MAE 0.107): двух значений хватает только для одной гармоники, а сумма двух гармоник требует p ≥ 4.
- При p ≥ 4 точность одинакова (MAE ≈ 0.024–0.027), а больший p лишь ускоряет обучение (2393 эпохи при p = 4, 132 при p = 6, 8 при p = 12).
- Ошибка перестаёт уменьшаться с p = 4.

2. Как число скрытых нейронов N влияет на точность и скорость обучения? Наблюдается ли переобучение (малая ошибка на обучении при заметно большей на тесте)? Достаточно ли минимального N, чтобы воспроизвести сумму двух гармоник?

- N почти не влияет ни на точность (MAE 0.0255–0.0273), ни на скорость (131–138 эпох).
- Переобучения нет: ошибки на обучении и тесте практически равны.
- Минимального N = 1 достаточно, потому что при малых весах сигмоида почти линейна, а ряд подчиняется линейному соотношению.

3. Почему на выходе для задачи прогнозирования используется линейная функция, а не сигмоида (как в ЛР №3)? Что произошло бы при сигмоиде на выходе и при обучении на ненормализованных значениях?

- В ЛР №3 (XOR) нужна была классификация с целями 0/1, поэтому подходила сигмоида. Здесь регрессия: прогноз может быть любым числом, а сигмоида ограничивает его интервалом (0; 1).
- Эксперимент с сигмоидой на выходе (нормализованные данные): за 3000 эпох ни один запуск не достиг Ee ($Es \approx 5 \cdot 10^{-4}$), а ошибка на тесте в 2–3 раза выше (MAE 0.06 против 0.026). Крайние значения ряда недостижимы, а вблизи 0 и 1 градиент затухает.
- Сигмоида на ненормализованных данных: цели лежат от −1.8 до 1.8, отрицательные значения она выдать не может. Es остаётся около 0.49, сеть не обучается.
- Линейный выход даёт такой же ряд без ограничений и сходится за 124–141 эпоху.

4. Почему нормализация должна опираться только на обучающий участок? К чему приводит выход тестовых значений за диапазон [0; 1] и как это проявляется на ваших данных?

- Min/max по всему ряду — это утечка информации из теста, в реальной задаче будущих данных нет.
- Две тестовые точки вышли за [0; 1] (k = 157 и 191), но почти не повлияли на ошибку.
- Сильнее это видно на выбросе: вход 2.18 дал прогноз −5.09 вместо −0.66.

5. Насколько устойчиво обучение к случайной инициализации весов (по сериям из 5 запусков)? Были ли запуски, не достигшие Ee за 20 000 эпох, и от чего это зависит (η, N, p)?

- Результат от инициализации зависит слабо (при p = 6 эпохи 118–146, σ MAE = 0.0009).
- Не сошлись только запуски с p = 2 (5 из 5): причина в слишком малом окне, а не в инициализации.
- Влияние η на p = 2 отдельно не проверялось.

6. Как растёт ошибка рекурсивного многошагового прогноза с увеличением горизонта? Объясните накопление ошибки и сравните одношаговый прогноз с многошаговым. До какого горизонта прогноз можно считать пригодным?

- Ошибка растёт волнами, максимум 0.159 на шаге 26.
- Средняя ошибка рекурсивного прогноза в 2.5 раза больше одношагового (0.052 против 0.021), потому что собственные ошибки подставляются в окно.
- По критерию 5 % размаха (0.18) прогноз пригоден на все 30 шагов.

7. Как согласуется различие между ошибкой на обучающем и тестовом участках с понятием обобщающей способности? Какие меры (ранняя остановка, уменьшение N, добавление шума, увеличение обучающей выборки) могли бы улучшить обобщение? Что показал эксперимент с шумом?

- Ошибки на обучении (0.0281) и на тесте (0.0277) почти равны, поэтому обобщающая способность хорошая.
- Улучшить обобщение могли бы ранняя остановка, меньшее N, больше данных.
- Шум σ = 0.02 увеличил тестовую MAE с 0.027 до 0.116, критерий Ee не достигнут ни в одном запуске (дисперсия шума $4 \cdot 10^{-4}$ уже больше $Ee = 10^{-4}$).